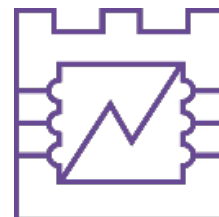




POLITECHNIKA KRAKOWSKA
im. T. Kościuszki

Wydział Inżynierii Elektrycznej i Komputerowej

Katedra



Kierunek studiów: Infotronika

Specjalność:

STUDIA STACJONARNE

PRACA DYPLOMOWA

MAGISTERSKA

Michał Filipiak

Projekt i implementacja serwerowego systemu automatyki domowej opartego na mikrokontrolerach ESP oraz protokole MQTT

Design and implementation of a server home automation system based on ESP microcontrollers and the MQTT protocol

Opiekun pracy:

dr inż. Paweł Król

Kraków, 2026

Spis treści

Streszczenie	4
Wykaz słów kluczowych	5
Wykaz ważniejszych oznaczeń i skrótów	6
1. Wstęp	7
1.1. Wprowadzenie	7
1.2. Cel i zakres pracy	7
1.3. Przegląd zawartości pracy	7
2. Podstawy teoretyczne	8
2.1. System Home Assistant	8
2.1.1. Filozofia i Architektura	8
2.1.2. Kluczowe Pojęcia i Model Danych	8
2.1.3. Silnik Automatyizacji	9
2.1.4. Ekosystem i Rozszerzalność	9
2.1.5. Interfejs Użytkownika i Komunikacja	9
2.2. Rozwiązania sprzętowe dla Home Assistant	10
2.2.1. Kontekst historyczny: Fundamenty informatyki	11
2.2.2. Komputery jednopłytkowe (SBC)	11
2.2.2.1. Raspberry Pi	11
2.2.2.2. ODROID-C4	12
2.2.3. Dedykowany sprzęt: Home Assistant Green	12
2.2.4. Rozwiązania wysokowydajne i klasy enterprise	13
2.2.4.1. Komputery osobiste x86-64	13
2.2.4.2. Nowoczesne komputery Mini PC: Apple Mac Mini	13
2.3. Protokół MQTT	14
2.3.1. Architektura i zasada działania	14
2.3.2. Mechanizmy zapewnienia jakości usług (QoS)	15
2.3.3. Kluczowe funkcje protokołu	15
2.3.4. MQTT w systemie Home Assistant	15
2.3.5. Bezpieczeństwo w MQTT	16
2.4. Docker	16
2.4.1. Konteneryzacja a Wirtualizacja	16
2.4.2. Architektura Systemu Docker	17
2.4.3. Kluczowe Pojęcia i Obiekty	17
2.4.4. Zalety Wykorzystania Dockera w Projektach Inżynierskich	18
3. Implementacja projektu i realizacja systemu	19
3.1. Ogólna architektura systemu	19
3.1.1. Model architektury systemu	19
3.1.2. Analiza przepływu danych	19
3.1.3. Uzasadnienie wyboru architektury	20
3.2. Przygotowanie środowiska serwerowego	20
3.2.1. Instalacja systemu operacyjnego Debian na laptopie Lenovo T470 . .	20

3.2.1.1.	Charakterystyka sprzętowa	20
3.2.1.2.	Przygotowanie nośnika instalacyjnego	21
3.2.1.3.	Konfiguracja BIOS/UEFI	21
3.2.1.4.	Przebieg instalacji	21
3.2.1.5.	Konfiguracja poinstalacyjna	22
3.2.2.	Konteneryzacja środowiska przy użyciu platformy Docker	22
3.2.2.1.	Instalacja silnika Docker Engine	22
3.2.2.2.	Konfiguracja poinstalacyjna	23
3.2.2.3.	Weryfikacja poprawności instalacji	23
3.2.2.4.	Wykorzystanie Docker Compose	23
3.3.	Uruchomienie brokera MQTT	23
3.3.1.	Uruchomienie brokera MQTT Mosquitto w kontenerze Docker	23
3.3.1.1.	Przygotowanie struktury katalogów	23
3.3.1.2.	Konfiguracja brokera Mosquitto	23
3.3.1.3.	Definicja usługi Docker Compose	24
3.3.1.4.	Uruchomienie i weryfikacja	24
3.4.	Postawienie Home Assistant	25
3.4.1.	Instalacja i konfiguracja platformy Home Assistant	25
3.4.1.1.	Wybór metody instalacji	25
3.4.1.2.	Konfiguracja usługi w Docker Compose	25
3.4.1.3.	Pierwsze uruchomienie i konfiguracja wstępna	25
3.4.1.4.	Integracja z brokerem MQTT	26
3.5.	Implementacja węzłów i analiza ruchu	26
3.5.1.	Implementacja oprogramowania węzła sensorowego (Node-1)	26
3.5.2.	Architektura i wybór komponentów	26
3.5.3.	Architektura i logika oprogramowania	27
3.5.4.	Proces przetwarzania i symulacji danych	27
3.5.5.	Integracja z Home Assistant	28
3.5.6.	Węzeł wykonawczy i silnik autowykrywania (Node-2)	28
3.5.7.	Architektura sprzętowa	28
3.5.8.	Separacja tematów: Polecenia a stan (Command vs State)	29
3.5.9.	Mechanizm MQTT Self-Discovery	29
3.5.10.	Logika operacyjna i synchronizacja	30
3.5.11.	Automatyzacja i zcentralizowane zarządzanie stanem	30
4.	Podsumowanie	32
	Bibliografia	33

Streszczenie

Wykaz słów kluczowych

Wykaz ważniejszych oznaczeń i skrótów

1. Wstęp

1.1. Wprowadzenie

W dzisiejszych czasach, wraz z dynamicznym rozwojem technologii i globalizacji, wiele dziedzin życia ulega znaczącym zmianom. Wprowadzenie nowych technologii, zmiany społeczne oraz rosnące wymagania stawiane przed jednostkami i organizacjami sprawiają, że konieczne jest ciągle dostosowywanie się do nowych warunków. W tym kontekście, niniejsza praca dyplomowa ma na celu zbadanie i analizę wybranych aspektów związanych z tymi zmianami, a także przedstawienie propozycji rozwiązań i rekomendacji, które mogą przyczynić się do lepszego zrozumienia i efektywnego radzenia sobie z wyzwaniami współczesnego świata.

1.2. Cel i zakres pracy

Celem niniejszej pracy dyplomowej jest...

1.3. Przegląd zawartości pracy

...

2. Podstawy teoretyczne

2.1. System Home Assistant

Home Assistant to zaawansowana platforma typu open-source, służąca do scentralizowanego zarządzania automatyką domową. System ten pełni rolę lokalnego kontrolera (tzw. bramki lub huba), integrującego różnorodne urządzenia i usługi w jedną, spójną strukturę. Kluczowym wyróżnikiem Home Assistant jest jego niezależność od dostawców chmurowych oraz nacisk na prywatność użytkownika i lokalne przetwarzanie danych.

Projekt został zapoczątkowany przez Paulusa Schoutsen'a we wrześniu 2013 roku jako aplikacja napisana w języku Python. Od tamtej pory, dzięki aktywnemu wsparciu społeczności zgromadzonej wokół serwisu GitHub, Home Assistant stał się jednym z najbardziej rozbudowanych i najczęściej aktualizowanych systemów automatyki domowej na świecie.

2.1.1. Filozofia i Architektura

Fundamentem Home Assistant jest idea „Local Control” (sterowanie lokalne) oraz „Privacy First” (prywatność przede wszystkim). W przeciwieństwie do wielu komercyjnych rozwiązań (np. Google Home, Amazon Alexa), Home Assistant dąży do tego, aby cała logika sterowania oraz przechowywanie danych odbywało się bezpośrednio w sieci lokalnej użytkownika. Takie podejście nie tylko zwiększa bezpieczeństwo, ale również zapewnia ciągłość działania systemu w przypadku awarii połączenia internetowego.

Architektura systemu opiera się na zdarzeniach (event-driven architecture). Wszystkie interakcje, zmiany stanów urządzeń czy wyzwalacze automatyzacji są procesowane przez centralną magistralę zdarzeń (Event Bus).

2.1.2. Kluczowe Pojęcia i Model Danych

W celu zachowania interoperacyjności między tysiącami różnych urządzeń, Home Assistant wykorzystuje sformalizowany model danych, którego podstawą są następujące komponenty:

- **Integracje (Integrations):** Są to moduły oprogramowania umożliwiające komunikację Home Assistant z zewnętrznymi platformami, urządzeniami lub usługami API. Na przykład integracja „Philips Hue” pozwala systemowi porozumiewać się z mostkiem Hue Bridge, a tym samym sterować oświetleniem tego producenta.
- **Urządzenia (Devices):** Reprezentują fizyczne lub logiczne obiekty w systemie. Jedno urządzenie (np. wielofunkcyjny czujnik) może składać się z wielu encji (np. czujnika temperatury, czujnika wilgotności i czujnika ruchu).
- **Encje (Entities):** To najmniejsze jednostki danych. Każda encja reprezentuje konkretną funkcję lub właściwość (np. stan włącznika, odczyt temperatury). Encje posiadają stany (States) oraz atrybuty, które są przechowywane w bazie danych systemu.
- **Obszary (Areas):** Pozwalają na logiczne grupowanie urządzeń i encji według ich fizycznej lokalizacji w domu (np. salon, kuchnia, sypialnia). Umożliwia to łatwiejsze

zarządzanie grupami urządzeń, np. wyłączenie wszystkich świateł v danym pomieszczeniu jedną komendą.

2.1.3. Silnik Automatykacji

Home Assistant oferuje potężny mechanizm automatyzacji, który pozwala na tworzenie złożonych scenariuszy interakcji między urządzeniami. Każda automatyzacja składa się z trzech podstawowych elementów:

1. **Wyzwalacze (Triggers):** Zdarzenia, które inicjują sprawdzenie automatyzacji (np. wykrycie ruchu, konkretna godzina, zachód słońca).
2. **Warunki (Conditions):** Opcjonalne kryteria, które muszą zostać spełnione, aby działanie zostało wykonane (np. „tylko jeśli jest po godzinie 22:00” lub „tylko jeśli nikogo nie ma w domu”).
3. **Działania (Actions):** Operacje wykonywane po spełnieniu warunków (np. włączenie światła, wysłanie powiadomienia na telefon).

Uzupełnieniem automatyzacji są **Skrypty**, będące sekwencjami działań wywoływanych na żądanie, oraz **Sceny**, które pozwalają na natychmiastowe przywrócenie określonego stanu wielu urządzeń jednocześnie (np. scena „Kino” przyciemniająca światła i zasłaniająca rolety).

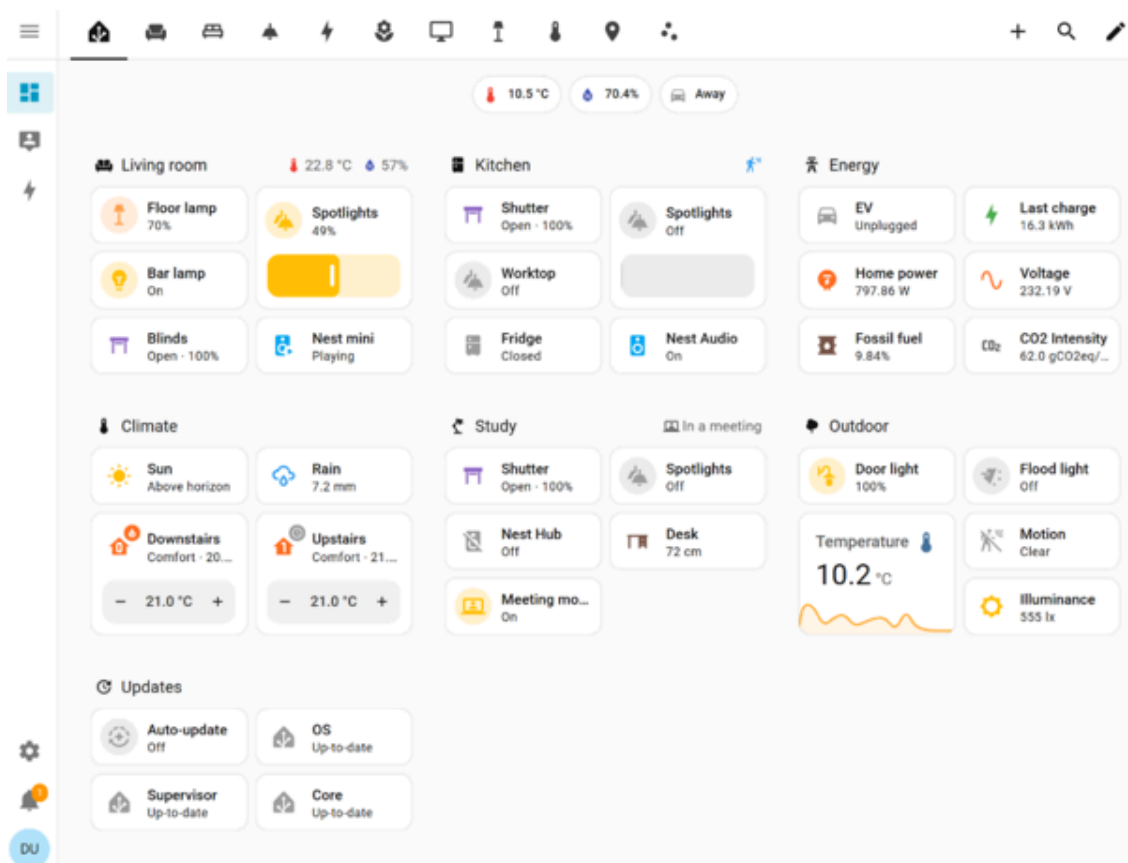
2.1.4. Ekosystem i Rozszerzalność

System Home Assistant wyróżnia się ogromnym ekosystemem, który znacznie wykracza poza standardowe integracje:

- **Dodatki (Add-ons):** To kontenery Dockerowe uruchamiane równolegle z Home Assistant, dostarczające dodatkowe usługi, takie jak serwer MQTT (Mosquitto), bazy danych (InfluxDB) czy narzędzia do wizualizacji (Grafana).
- **HACS (Home Assistant Community Store):** Nieoficjalne repozytorium społecznościowe, pozwalające na instalację niestandardowych integracji oraz elementów interfejsu graficznego, które nie zostały jeszcze włączone do oficjalnej dystrybucji.
- **Blueprints:** Gotowe szablony automatyzacji, które użytkownicy mogą łatwo importować i konfigurować bez konieczności samodzielnego tworzenia logiki od podstaw.

2.1.5. Interfejs Użytkownika i Komunikacja

Głównym sposobem interakcji z systemem jest pulpit nawigacyjny (Dashboard), oparty na silniku Lovelace. Jest on w pełni konfigurowalny i responsywny, co pozwala na tworzenie przejrzystych interfejsów sterowania dostępnych zarówno przez przeglądarkę WWW, jak i dedykowane aplikacje mobilne na systemy Android i iOS. Przykładowy wygląd pulpitu nawigacyjnego przedstawiono na rysunku Rysunek 1.



Rysunek 1: Przykładowy pulpit nawigacyjny systemu Home Assistant

Home Assistant wspiera szeroki wachlarz protokołów komunikacyjnych, co czyni go prawdziwym centrum inteligentnego domu. Do najpopularniejszych należą:

- **Wi-Fi:** Wykorzystywany przez wiele urządzeń IoT, często poprzez protokół MQTT.
- **Zigbee i Z-Wave:** Energooszczędne protokoły typu mesh, idealne dla czujników bateryjnych.
- **Matter:** Najnowszy standard interoperacyjności, wspierany przez największych graczy rynkowych, którego Home Assistant był jednym z wczesnych propagatorów.

Dzięki swojej elastyczności i otwartości, Home Assistant jest obecnie uznawany za jedną z najbardziej wszechstronnych platform do budowy zaawansowanych systemów Smart Home, stanowiąc doskonałą bazę do badań naukowych i realizacji projektów inżynierskich.

2.2. Rozwiązania sprzętowe dla Home Assistant

Wybór odpowiedniej platformy sprzętowej jest fundamentalną decyzją przy wdrażaniu systemu automatyki domowej opartego na Home Assistant. Jako platforma open-source, Home Assistant został zaprojektowany tak, aby był niezależny od sprzętu, wspierając szeroką gamę architektur — od energooszczędnych komputerów jednopłytkowych opartych na architekturze ARM po wysokowydajne serwery x86-64. Wybór zależy od kilku czynników: złożoności logiki automatyzacji, liczby zintegrowanych urządzeń, potrzeby lokalnego przetwarzania danych (takich jak rozpoznawanie obrazu) oraz pożądanego poziomu niezawodności i redundancji systemu.

2.2.1. Kontekst historyczny: Fundamenty informatyki

Aby zrozumieć obecny krajobraz sprzętu do automatyki domowej, warto spojrzeć wstecz na początki komputerów osobistych. Procesor Intel 8086, wprowadzony w 1978 roku, ustanowił architekturę x86, która do dziś pozostaje dominującą siłą w informatyce. Choć sam 8086 jest zbyt prymitywny dla nowoczesnej automatyki domowej, jego dziedzictwo żyje w potężnych serwerach x86-64, których wielu entuzjastów używa do uruchamiania złożonych instancji Home Assistant.



Rysunek 2: Procesor Intel 8086, który zapoczątkował architekturę wciąż używaną w nowoczesnych, wysokiej klasy serwerach domowych.

2.2.2. Komputery jednopłytkowe (SBC)

Komputery jednopłytkowe stały się najpopularniejszym wyborem dla użytkowników Home Assistant ze względu na niski pobór mocy, kompaktowe rozmiary i cichą pracę.

2.2.2.1. Raspberry Pi

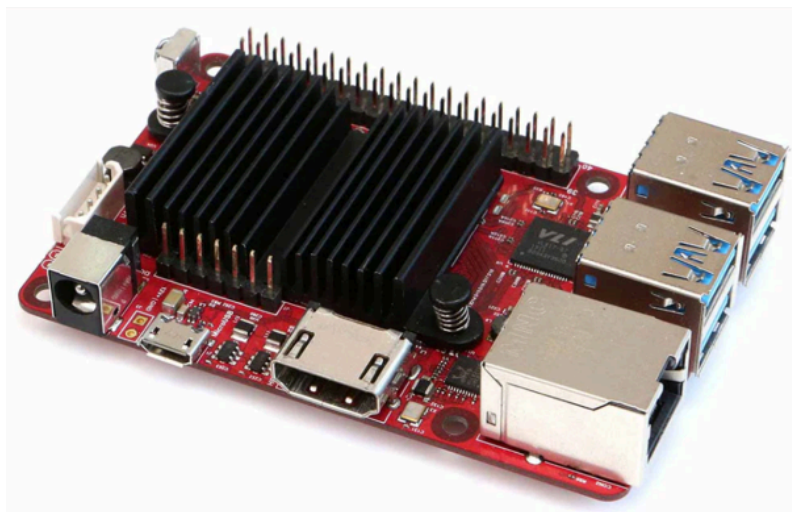
Seria Raspberry Pi jest najszerzej wspieraną platformą dla Home Assistant. Jej popularność wynika z przystępnej ceny i ogromnego wsparcia społeczności. System „Home Assistant OS” (HAOS) jest specjalnie zoptymalizowany dla Raspberry Pi, oferując użytkownikom doświadczenie typu „po prostu działa”.



Rysunek 3: Raspberry Pi pozostaje standardem wejściowym dla entuzjastów automatyki domowej.

2.2.2.2. ODROID-C4

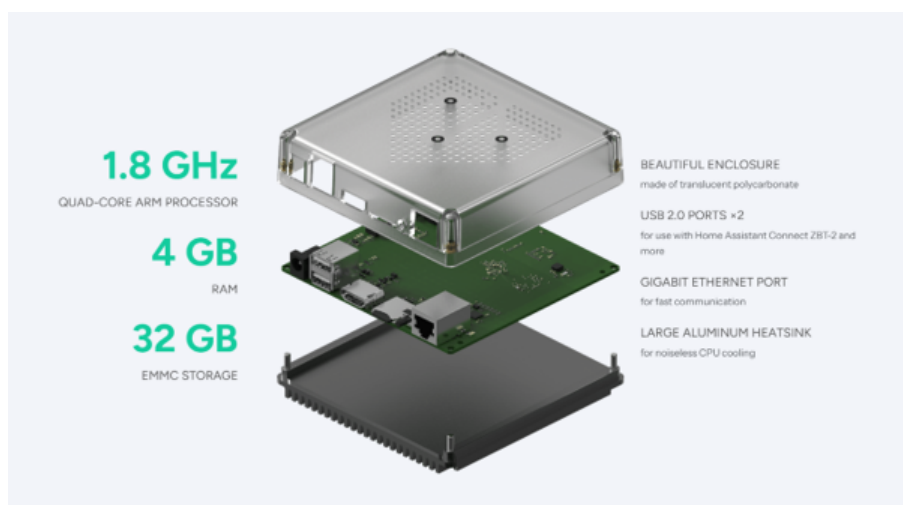
Dla użytkowników wymagających większej stabilności i lepszej wydajności pamięci masowej niż w przypadku konstrukcji Raspberry Pi opartej na kartach microSD, ODROID-C4 stanowi przekonującą alternatywę. Dzięki obsłudze modułów eMMC, ODROID-C4 zapewnia znacznie wyższą niezawodność operacji wejścia/wyjścia dysku, co jest kluczowe dla baz danych SQLite lub MariaDB, których Home Assistant używa do przechowywania historycznych danych o stanie urządzeń.



Rysunek 4: ODROID-C4 stanowi solidną alternatywę z lepszymi opcjami pamięci masowej, takimi jak eMMC.

2.2.3. Dedykowany sprzęt: Home Assistant Green

Dostrzegając potrzebę prostego punktu wejścia, Nabu Casa (firma stojąca za Home Assistant) wprowadziła Home Assistant Green. Urządzenie to zostało zaprojektowane jako najłatwiejszy sposób na rozpoczęcie przygody z Home Assistant, dostarczane z preinstalowanym oprogramowaniem i zoptymalizowane pod kątem wydajności od razu po wyjęciu z pudełka, bez konieczności montażu technicznego.



Rysunek 5: Widok architektoniczny 3D wewnętrznych komponentów sprzętowych Home Assistant Green, prezentujący jego zoptymalizowaną konstrukcję.

2.2.4. Rozwiązania wysokowydajne i klasy enterprise

W miarę jak systemy automatyki domowej rozrastają się i obejmują dziesiątki kamer, złożone modele uczenia maszynowego do wykrywania obiektów oraz liczne usługi zwirtualizowane, ograniczenia komputerów SBC stają się widoczne. W takich przypadkach użytkownicy zwracają się ku mocniejszemu sprzętowi.

2.2.4.1. Komputery osobiste x86-64

Wykorzystanie standardowego komputera PC x86-64 lub dedykowanego serwera pozwala na praktycznie nieograniczoną rozbudowę. Systemy te obsługują szybkie pamięci masowe NVMe, duże ilości pamięci RAM i potężne procesory zdolne do obsługi intensywnych zadań, takich jak transkodowanie wideo w czasie rzeczywistym i automatyzacja oparta na sztucznej inteligencji.



Rysunek 6: Standardowy komputer PC x86-64 oferuje maksymalną elastyczność i wydajność dla instalacji na dużą skalę.

2.2.4.2. Nowoczesne komputery Mini PC: Apple Mac Mini

Najnowsza generacja komputerów Mini PC, której przykładem jest Apple Mac Mini z czipem M4, reprezentuje szczyt wydajności w stosunku do zużytej energii. Urządzenia te oferują wydajność klasy stacji roboczej w obudowie niewiele większej od niektórych komputerów SBC. Dla użytkowników, którzy chcą uruchamiać Home Assistant obok innych ciężkich obciążeń (takich jak serwer mediów czy prywatna chmura), te nowoczesne jednostki zapewniają doskonałą równowagę między mocą a wydajnością.



Rysunek 7: Apple Mac Mini M4 służy jako wysokiej klasy, energooszczędna platforma dla wymagających centrów automatyki domowej.

Krajobraz sprzętowy dla Home Assistant jest zróżnicowany, od przystępnych cenowo Raspberry Pi i Home Assistant Green po wysokowydajne serwery x86-64 i nowoczesne komputery Mini PC. Idealny wybór jest określany przez specyficzne wymagania ekosystemu inteligentnego domu, balansując między kosztem, łatwością obsługi a zapotrzebowaniem na moc obliczeniową.

2.3. Protokół MQTT

Protokół MQTT (ang. **Message Queuing Telemetry Transport**) jest jednym z najpopularniejszych standardów komunikacyjnych wykorzystywanych w świecie Internetu Rzeczy (IoT). Został opracowany w 1999 roku przez Andy'ego Stanforda-Clarka (IBM) oraz Arlena Nipperera (Arcom) z myślą o monitorowaniu rurociągów naftowych za pośrednictwem łącz satelitarnych. Głównym założeniem projektowym była minimalizacja narzutu danych przy jednoczesnym zapewnieniu niezawodności w sieciach o niskiej przepustowości i dużej latencji.

Obecnie MQTT jest standardem otwartym, zarządzanym przez organizację OASIS. Najpowszechniej stosowaną wersją jest 3.1.1, jednak coraz większą popularność zyskuje wersja 5.0, wprowadzająca m.in. ulepszone raportowanie błędów oraz metadane wiadomości (ang. **User Properties**). Lekkość i wydajność protokołu sprawiły, że stał się on de facto standardem w systemach automatyki domowej, w tym w szczególności w ekosystemie Home Assistant.

2.3.1. Architektura i zasada działania

W przeciwieństwie do tradycyjnego modelu klient-serwer (opartego na żądaniach HTTP), MQTT wykorzystuje model **publikacja-subskrypcja** (ang. **publish-subscribe**). W architekturze tej urządzenia końcowe nie komunikują się bezpośrednio ze sobą, lecz za pośrednictwem centralnego węzła zwanego **brokerem**.

Kluczowe komponenty architektury MQTT to:

- **Broker:** Centralny serwer odpowiedzialny za odbieranie wszystkich wiadomości, ich filtrowanie oraz dystrybucję do odpowiednich odbiorców. W kontekście Home Assistant najczęściej wykorzystywanym brokerem jest **Eclipse Mosquitto**.
- **Klient:** Dowolne urządzenie (np. czujnik temperatury, inteligentne gniazdko, smartfon) lub aplikacja, która nawiązuje połączenie z brokerem. Klient może pełnić rolę publikującego, subskrybującego lub obie te role jednocześnie.
- **Temat (Topic):** Ciąg znaków o strukturze hierarchicznej (oddzielanej ukośnikami, np. dom/salon/temperatura), który służy jako adres logiczny wiadomości. Broker wykorzystuje tematy do decydowania, do których klientów ma trafić dana informacja.

Zasada działania polega na tym, że klient subskrybujący informuje brokera o chęci otrzymywania wiadomości z określonych tematów. Gdy inny klient (publikujący) wyśle wiadomość do danego tematu, broker natychmiast przekazuje ją do wszystkich klientów, którzy go subskrybują.

2.3.2. Mechanizmy zapewnienia jakości usług (QoS)

Jedną z najistotniejszych cech MQTT jest możliwość definiowania poziomu jakości usług (ang. **Quality of Service** – QoS), co pozwala na dostosowanie niezawodności dostarczania wiadomości do wymagań konkretnego zastosowania:

- **QoS 0 (At most once):** Wiadomość jest wysyłana tylko raz, bez potwierdzenia odbioru. Jest to najszybszy tryb, ale nie gwarantuje dostarczenia danych (np. dla cyklicznych odczytów temperatury, gdzie utrata jednego pomiaru nie jest krytyczna).
- **QoS 1 (At least once):** Gwarantuje, że wiadomość dotrze do odbiorcy przynajmniej raz. Wymaga potwierdzenia odbioru, a w przypadku jego braku następuje retransmisja.
- **QoS 2 (Exactly once):** Najbardziej zaawansowany i kosztowny pod względem narzutu sieciowego tryb, gwarantujący, że wiadomość dotrze do odbiorcy dokładnie jeden raz, bez duplikacji.

2.3.3. Kluczowe funkcje protokołu

Protokół MQTT oferuje szereg mechanizmów optymalizujących komunikację w systemach inteligentnego domu:

- **Retained Messages (Wiadomości utrwalone):** Broker przechowuje ostatnią poprawną wiadomość wysłaną na dany temat. Dzięki temu nowy klient, tuż po subskrypcji tematu, natychmiast otrzymuje aktualny stan urządzenia, nie czekając na kolejną publikację.
- **Last Will and Testament (LWT):** Pozwala klientowi zdefiniować wiadomość, która zostanie automatycznie opublikowana przez brokera w przypadku nieoczekiwanego rozłączenia klienta (np. utraty zasilania lub zasięgu Wi-Fi). W Home Assistant mechanizm ten jest kluczowy dla oznaczania urządzeń jako niedostępne (**unavailable**).
- **Keep Alive:** Mechanizm cyklicznego sprawdzania połączenia (ping), który pozwala brokerowi i klientowi na szybkie wykrycie zerwania sesji.

2.3.4. MQTT w systemie Home Assistant

W systemie Home Assistant protokół MQTT pełni rolę uniwersalnej magistrali komunikacyjnej, umożliwiającej integrację urządzeń od różnych producentów oraz własnych rozwiązań opartych na mikrokontrolerach (np. ESP8266, ESP32).

Szczególne znaczenie ma mechanizm **MQTT Discovery**. Pozwala on urządzeniom na automatyczne przedstawienie się systemowi Home Assistant poprzez wysłanie specjalnie sformatowanej wiadomości konfiguracyjnej w formacie JSON na określony temat (domyślnie `homeassistant/`). Dzięki temu użytkownik nie musi ręcznie edytować plików konfiguracyjnych – nowe urządzenie pojawia się w interfejsie automatycznie jako gotowa do użycia encja.

Najpopularniejszym sposobem implementacji MQTT w Home Assistant jest instalacja dodatku **Mosquitto broker**, który jest w pełni zintegrowany z systemem.

2.3.5. Bezpieczeństwo w MQTT

Ze względu na to, że MQTT często przesyła dane w sieciach lokalnych i rozległych, kluczowe jest zapewnienie odpowiedniego poziomu bezpieczeństwa. Protokół wspiera:

- **Uwierzytelnianie:** Wymaganie nazwy użytkownika i hasła przy nawiązywaniu połączenia z brokerem.
- **Autoryzację (ACL):** Definiowanie uprawnień dla poszczególnych użytkowników (np. dany klient może tylko subskrybować konkretny temat, ale nie może na nim publikować).
- **Szyfrowanie TLS/SSL:** Zabezpieczenie całej komunikacji przed podsłuchem i atakami typu **Man-in-the-Middle**.

Wykorzystanie MQTT w połączeniu z oprogramowaniem takim jak **Tasmota**, **Zigbee2MQTT** czy **ESPHome** stanowi fundament zaawansowanych i stabilnych instalacji inteligentnego domu.

2.4. Docker

Docker to otwarta platforma służąca do automatyzacji wdrażania, skalowania i zarządzania aplikacjami wewnątrz lekkich i przenośnych kontenerów. Została ona zapoczątkowana przez Solomona Hykesa i udostępniona jako projekt open-source w marcu 2013 roku. Docker zrewolucjonizował podejście do tworzenia oprogramowania, wprowadzając standard konteneryzacji, który pozwala na izolację aplikacji od infrastruktury systemowej, zapewniając ich spójne działanie w różnych środowiskach — od lokalnej maszyny programisty po zaawansowane klastry obliczeniowe w chmurze.

2.4.1. Konteneryzacja a Wirtualizacja

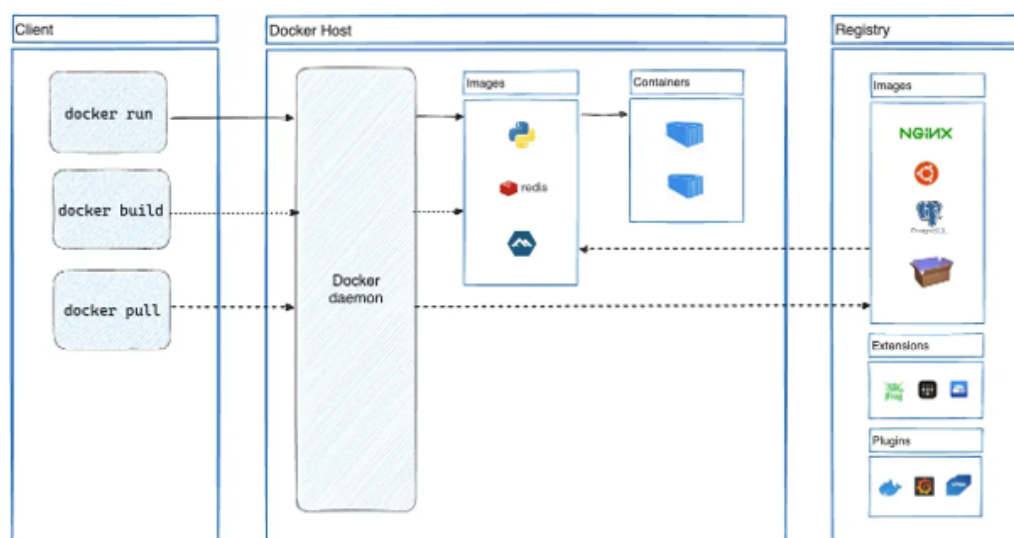
Kluczem do zrozumienia fenomenu Dockera jest rozróżnienie między tradycyjną wirtualizacją a konteneryzacją. Tradycyjne maszyny wirtualne (VM) wymagają pełnego systemu operacyjnego (Guest OS) dla każdej instancji, co wiąże się z dużym narzutem zasobów (procesora, pamięci RAM i miejsca na dysku).

Konteneryzacja, na której opiera się Docker, wykorzystuje mechanizmy jądra systemu operacyjnego gospodarza (takie jak namespaces i control groups w systemie Linux) do izolacji procesów. Dzięki temu kontenery współdzielą jądro systemu z gospodarzem, co czyni je znacznie lżejszymi, szybszymi w uruchamianiu i bardziej wydajnymi pod względem wykorzystania zasobów niż maszyny wirtualne.

2.4.2. Architektura Systemu Docker

Architektura Dockera opiera się na modelu klient-serwer (rysunek Rysunek 8). Głównymi elementami tego systemu są:

- **Docker Daemon (dockerd):** Działa w tle na systemie gospodarza i zarządza obiektami Dockera, takimi jak obrazy, kontenery, sieci oraz wolumeny.
- **Docker Client:** Narzędzie linii komend (CLI), które pozwala użytkownikowi komunikować się z demonem Dockera poprzez API REST.
- **Docker Registry:** Miejsce przechowywania obrazów. Najpopularniejszym publicznym rejestrem jest Docker Hub, ale organizacje często korzystają z prywatnych rejestrów w celu zapewnienia bezpieczeństwa.



Rysunek 8: Architektura platformy Docker i interakcja między jej komponentami

2.4.3. Kluczowe Pojęcia i Obiekty

W ekosystemie Dockera wyróżnia się kilka fundamentalnych pojęć, które stanowią o jego funkcjonalności:

- **Obraz (Image):** Tylko do odczytu szablon instrukcji służący do tworzenia kontenera. Często obraz bazuje na innym obrazie z dodatkowymi modyfikacjami (np. obraz aplikacji bazujący na obrazie systemu Ubuntu z zainstalowanym serwerem Apache).
- **Kontener (Container):** Uruchomiona instancja obrazu. Można go tworzyć, uruchamiać, zatrzymywać, przenosić lub usuwać. Kontenery są z definicji ulotne, co oznacza, że dane w nich zapisane zostają utracone po ich usunięciu, chyba że zostaną użyte mechanizmy trwałości danych (wolumeny).
- **Dockerfile:** Plik tekstowy zawierający zestaw instrukcji, które Docker wykonuje w celu automatycznego zbudowania obrazu. Pozwala to na pełną dokumentację środowiska i łatwą reprodukcję artefaktów.
- **Wolumeny (Volumes):** Mechanizm służący do trwałego przechowywania danych generowanych i używanych przez kontenery. Wolumeny są zarządzane przez Dockera i są niezależne od cyklu życia samego kontenera.

2.4.4. Zalety Wykorzystania Dockera w Projektach Inżynierskich

Zastosowanie Dockera w procesie wytwórczym oprogramowania niesie ze sobą szereg korzyści:

1. **Izolacja środowiska:** Każdy kontener posiada własne biblioteki i zależności, co eliminuje problem „u mnie działa” oraz konflikty między różnymi wersjami oprogramowania na tej samej maszynie.
2. **Przenośność:** Raz zbudowany obraz może zostać uruchomiony na dowolnym systemie wspierającym Dockera, bez konieczności dodatkowej konfiguracji.
3. **Szybkość i wydajność:** Kontenery uruchamiają się w ciągu milisekund, co pozwala na błyskawiczne skalowanie usług i optymalne wykorzystanie infrastruktury.
4. **Usprawnienie CI/CD:** Docker idealnie wpisuje się w potoki ciągłej integracji i ciągłego wdrażania, umożliwiając automatyczne budowanie i testowanie identycznych artefaktów na każdym etapie cyklu życia aplikacji.

Dzięki tym cechom Docker stał się standardem de facto w nowoczesnym inżynierii oprogramowania, stanowiąc fundament dla architektury mikroserwisowej oraz nowoczesnych systemów automatyki i IoT.

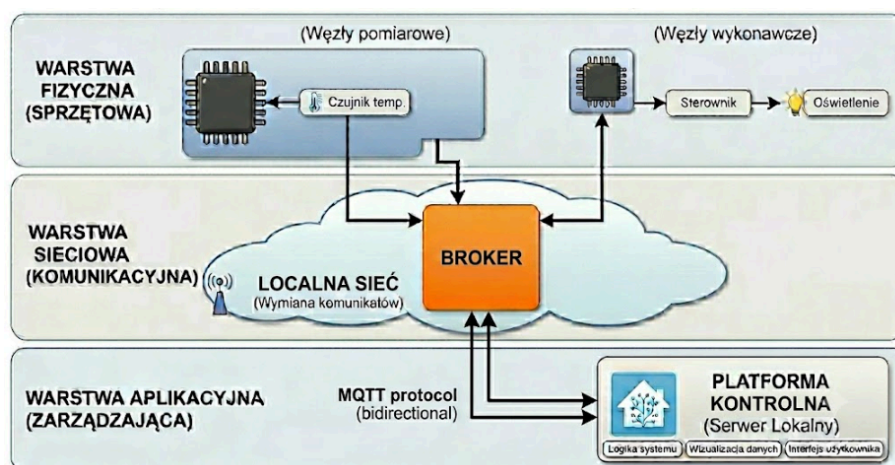
3. Implementacja projektu i realizacja systemu

3.1. Ogólna architektura systemu

W niniejszym rozdziale przedstawiono szczegółowy opis implementacji systemu automatyki domowej, opartego na rozproszonej architekturze z wykorzystaniem protokołu MQTT oraz platformy Home Assistant. Przed przystąpieniem do opisu poszczególnych etapów konfiguracji, konieczne jest zrozumienie ogólnej struktury systemu oraz sposobu przepływu danych między jego komponentami.

3.1.1. Model architektury systemu

Projektowany system opiera się na trójwarstwowym modelu architektury rozproszonej, który zapewnia skalowalność oraz łatwość w rozbudowie o kolejne węzły sensorowe i wykonawcze. Na rysunku X przedstawiono schemat ideowy połączeń między kluczowymi elementami systemu.



Rysunek 9: Ogólny schemat architektury systemu.

Wyróżnić można następujące warstwy:

1. **Warstwa fizyczna (sprzętowa):** Składa się z mikrokontrolerów rodziny ESP32, które pełnią rolę węzłów pomiarowych (zbieranie danych z czujników temperatury, wilgotności itp.) oraz węzłów wykonawczych (sterowanie przekaźnikami, oświetleniem).
2. **Warstwa sieciowa (komunikacyjna):** Wykorzystuje lokalną sieć Wi-Fi oraz centralny broker MQTT (Mosquitto), który pośredniczy w wymianie komunikatów między urządzeniami a systemem nadrzędnym.
3. **Warstwa aplikacyjna (zarządzająca):** Stanowi ją platforma Home Assistant zainstalowana na serwerze lokalnym, która odpowiada za logikę systemu, wizualizację danych oraz interfejs użytkownika.

3.1.2. Analiza przepływu danych

Komunikacja w systemie odbywa się w sposób asynchroniczny, co pozwala na minimalizację zużycia energii przez węzły końcowe oraz zwiększenie niezawodności. Przepływ danych można podzielić na dwa główne kierunki:

- **Dane telemetryczne (Upstream):** Węzły sensorowe cyklicznie przesyłają odczyty z czujników do brokera MQTT pod odpowiednie tematy (np. `home/livingroom/temperature`). System Home Assistant, subskrybując te tematy, odbiera dane, przetwarza je i zapisuje w bazie danych.
- **Polecenia sterujące (Downstream):** Użytkownik poprzez interfejs Home Assistant lub automatyzację generuje polecenie (np. włączenie światła). Home Assistant publikuje odpowiedni komunikat na brokerze MQTT (np. `home/livingroom/light/set`), który jest natychmiast odbierany przez właściwy węzeł wykonawczy.

3.1.3. Uzasadnienie wyboru architektury

Wybór architektury opartej na brokerze MQTT, zamiast bezpośredniej komunikacji HTTP (Peer-to-Peer), był podyktowany kilkoma kluczowymi czynnikami:

- **Luźne powiązanie komponentów (Decoupling):** Urządzenia końcowe nie muszą znać adresu IP serwera Home Assistant ani jego stanu. Komunikują się jedynie z brokerem, co ułatwia wymianę lub dodawanie nowych elementów systemu bez konieczności rekonfiguracji pozostałych.
- **Odporność na błędy:** W przypadku chwilowej niedostępności systemu nadrzędnego, broker może przechowywać ostatnie komunikaty (funkcja Retain) lub zarządzać sesjami urządzeń.
- **Niskie narzuty komunikacyjne:** Protokół MQTT jest znacznie lżejszy od HTTP, co jest kluczowe przy wykorzystaniu mikrokontrolerów o ograniczonych zasobach oraz w celu zapewnienia niskich opóźnień w działaniu automatyzacji.

3.2. Przygotowanie środowiska serwerowego

3.2.1. Instalacja systemu operacyjnego Debian na laptopie Lenovo T470

Pierwszym etapem prac praktycznych było przygotowanie środowiska systemowego. Zdecydowano się na wybór dystrybucji Debian GNU/Linux w wersji 12 (Bookworm), która charakteryzuje się wysoką stabilnością oraz szerokim wsparciem społeczności, co jest kluczowe w projektach o charakterze badawczym i inżynierskim.

3.2.1.1. Charakterystyka sprzętowa

Proces instalacji przeprowadzono na laptopie Lenovo ThinkPad T470. Model ten jest powszechnie ceniony za wysoką kompatybilność z systemami z rodziny Linux. Kluczowe parametry techniczne wykorzystanego urządzenia zestawiono poniżej:

- Procesor: Intel Core i5-7300U,
- Pamięć RAM: 16 GB DDR4,
- Dysk: 256 GB SSD NVMe,
- Karta sieciowa: Intel Dual Band Wireless-AC 8265.



Rysunek 10: Widok ogólny laptopa Lenovo ThinkPad T470 wykorzystanego w badaniach.

3.2.1.2. Przygotowanie nośnika instalacyjnego

Proces rozpoczęto od pobrania oficjalnego obrazu ISO typu „netinst” z serwerów projektu Debian. Wykorzystanie obrazu sieciowego umożliwiło instalację najnowszych wersji pakietów już na etapie wdrażania systemu. Obraz został przeniesiony na nośnik USB o pojemności 16 GB przy użyciu systemowego narzędzia dd w środowisku Linux. Operację wykonano przy użyciu następującego polecenia:

```
sudo dd if=debian-12-netinst-amd64.iso of=/dev/sdX bs=4M status=progress  
oflag=sync
```

3.2.1.3. Konfiguracja BIOS/UEFI

Przed przystąpieniem do właściwej instalacji konieczne było dostosowanie ustawień oprogramowania układowego (UEFI) urządzenia. Kluczowe modyfikacje objęły:

1. Wyłączenie funkcji **Secure Boot**, co zapewnia bezproblemowe ładowanie modułów jądra.
2. Ustawienie trybu pracy kontrolera pamięci masowej na **AHCI**.
3. Zmianę priorytetu urządzeń rozruchowych, ustawiając nośnik USB jako nadrzędny.

3.2.1.4. Przebieg instalacji

Po uruchomieniu komputera z przygotowanego nośnika wybrano tryb **Graphical Install**. Proces przebiegał zgodnie z następującymi etapami:

1. Konfiguracja regionalna: Wybrano język polski oraz układ klawiatury programisty.
2. Konfiguracja sieci: Dzięki zintegrowaniu w jądrze systemu Debian 12 sterowników z sekcji **non-free-firmware**, karta sieciowa Intel została poprawnie zidentyfikowana, co umożliwiło nawiązanie połączenia bezprzewodowego w trakcie instalacji.
3. Partycjonowanie dysku: Zastosowano automatyczne partycjonowanie całej przestrzeni dyskowej z wykorzystaniem mechanizmu LVM (Logical Volume Manager). Wybór ten podyktowany był elastycznością w przyszłym zarządzaniu wolumenami logicznymi.
4. Wybór oprogramowania: Zdecydowano o pominięciu instalacji środowiska graficznego. Wybrano jedynie podstawowe narzędzia systemowe (**Standard system utilities**) oraz serwer SSH (**SSH server**). Takie podejście pozwoliło na minimalizację narzutu systemowego i zwiększenie poziomu bezpieczeństwa stacji roboczej.

3.2.1.5. Konfiguracja poinstalacyjna

Po pomyślnym zakończeniu procesu i ponownym uruchomieniu urządzenia, przeprowadzono kroki konfiguracyjne:

- Dodanie konta użytkownika do grupy `sudo`, umożliwiając zarządzanie systemem z uprawnieniami administratora.
- Weryfikacja pliku `/etc/apt/sources.list` pod kątem obecności repozytoriów `main`, `contrib` oraz `non-free-firmware`.
- Wykonanie pełnej aktualizacji pakietów systemowych poleceniem `sudo apt update && sudo apt upgrade`.

System Debian 12 na platformie Lenovo T470 wykazał pełną kompatybilność sprzętową bezpośrednio po instalacji, co stanowi stabilny fundament dla dalszych prac badawczych.

3.2.2. Konteneryzacja środowiska przy użyciu platformy Docker

Po skonfigurowaniu systemu operacyjnego Debian, kolejnym krokiem było wdrożenie platformy Docker. Konteneryzacja pozwala na izolację poszczególnych usług (takich jak broker MQTT czy Home Assistant), co znacząco ułatwia zarządzanie zależnościami oraz migrację systemu między różnymi platformami sprzętowymi.

3.2.2.1. Instalacja silnika Docker Engine

Proces instalacji przeprowadzono zgodnie z oficjalną dokumentacją, wykorzystując repozytoria producenta w celu zapewnienia dostępu do najnowszych wersji oprogramowania.

1. **Aktualizacja indeksu pakietów i instalacja zależności:** W pierwszej kolejności zainstalowano pakiety umożliwiające bezpieczną komunikację z repozytoriami przez protokół HTTPS:

```
sudo apt update
sudo apt install ca-certificates curl gnupg
```

2. **Dodanie oficjalnego klucza GPG Docker:** Klucz służy do weryfikacji autentyczności pobieranych pakietów:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

3. **Konfiguracja repozytorium:** Dodano wpis do listy źródeł systemowego menedżera pakietów `apt`:

```
echo \
"deb [arch="$(dpkg --print-architecture)" signed-by=/etc/apt/
keyrings/docker.gpg] https://download.docker.com/linux/debian \
"${(. /etc/os-release && echo "$VERSION_CODENAME")}" stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

4. **Instalacja pakietów Docker:** Po zaktualizowaniu listy repozytoriów, zainstalowano silnik Docker oraz wtyczkę Docker Compose:

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

3.2.2.2. Konfiguracja poinstalacyjna

Domyślnie demon Docker wymaga uprawnień administratora. Aby umożliwić zarządzanie kontenerami bez każdorazowego użycia polecenia `sudo`, dodano bieżącego użytkownika do grupy `docker`:

```
sudo usermod -aG docker $USER
```

Zmiana ta wymagała ponownego zalogowania się do systemu, aby nowa przynależność do grupy została uwzględniona w bieżącej sesji.

3.2.2.3. Weryfikacja poprawności instalacji

Poprawność wdrożenia zweryfikowano poprzez uruchomienie testowego kontenera `hello-world`:

```
docker run hello-world
```

Pomyślne wykonanie tego polecenia potwierdziło, że silnik Docker jest poprawnie zainstalowany, a użytkownik posiada niezbędne uprawnienia do zarządzania kontenerami.

3.2.2.4. Wykorzystanie Docker Compose

W dalszych etapach pracy, zamiast pojedynczych poleceń `docker run`, zdecydowano się na wykorzystanie narzędzia Docker Compose. Pozwala ono na definiowanie całej infrastruktury w formie deklaracyjnych plików `docker-compose.yml`, co zapewnia powtarzalność konfiguracji i ułatwia dokumentowanie wdrożonych usług.

3.3. Uruchomienie brokera MQTT

3.3.1. Uruchomienie brokera MQTT Mosquitto w kontenerze Docker

Kolejnym kluczowym elementem infrastruktury jest broker MQTT (Message Queuing Telemetry Transport), który pełni rolę centralnego węzła komunikacyjnego w systemie automatyki budynkowej. Wybrano oprogramowanie Eclipse Mosquitto ze względu na jego lekkość, wysoką wydajność oraz pełną zgodność ze standardem MQTT.

3.3.1.1. Przygotowanie struktury katalogów

Zgodnie z dobrą praktyką zarządzania kontenerami, dla usługi brokera utworzono dedykowany katalog roboczy `~/mqtt`, w którym przechowywane są pliki konfiguracyjne oraz dane trwałe (persistence). Pozwala to na separację danych aplikacji od samego obrazu kontenera.

```
mkdir -p ~/mqtt/config ~/mqtt/data ~/mqtt/log
cd ~/mqtt
```

3.3.1.2. Konfiguracja brokera Mosquitto

Podstawowa konfiguracja brokera została zdefiniowana w pliku `~/mqtt/config/mosquitto.conf`. W celu zapewnienia poprawnej komunikacji wewnątrz sieci kontenerowej oraz dostępu z zewnątrz, przyjęto następujące ustawienia:

```
# mosquitto.conf
persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log

listener 1883
allow_anonymous true
```

Powyższa konfiguracja włącza trwałość danych (zachowywanie stanu subskrypcji i wiadomości po restarcie), definiuje ścieżki do logów oraz uruchamia nasłuchiwanie na standardowym porcie 1883. Na etapie testowym zezwolono na połączenia anonimowe (`allow_anonymous true`), co upraszcza wstępną weryfikację poprawności działania systemu bez konieczności definiowania list kontroli dostępu (ACL).

3.3.1.3. Definicja usługi Docker Compose

Do uruchomienia brokera wykorzystano narzędzie Docker Compose, co pozwala na deklaratywne zdefiniowanie parametrów kontenera. Plik `docker-compose.yml` został skonfigurowany w następujący sposób:

```
version: '3.8'

services:
  mosquitto:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: always
    ports:
      - "1883:1883"
      - "9001:9001"
    volumes:
      - ./config:/mosquitto/config
      - ./data:/mosquitto/data
      - ./log:/mosquitto/log
```

Mapowanie wolumenów (`volumes`) zapewnia, że pliki konfiguracyjne oraz zebrane dane są mapowane bezpośrednio z systemu gospodarza do wnętrza kontenera. Port 1883 służy do standardowej komunikacji MQTT, natomiast port 9001 jest zarezerwowany dla komunikacji poprzez WebSockets.

3.3.1.4. Uruchomienie i weryfikacja

Uruchomienie brokera odbywa się za pomocą jednego polecenia wykonanego w katalogu roboczym:

```
docker-compose up -d
```

W celu weryfikacji poprawności działania usługi sprawdzono status kontenera oraz przeanalizowano logi systemowe. Do testów funkcjonalnych wykorzystano narzędzia klienckie `mosquitto_sub` i `mosquitto_pub`. W jednym oknie terminala uruchomiono subskrypcję tematu testowego:

```
mosquitto_sub -h localhost -t "test/topic"
```


W drugim terminalu przesłano wiadomość testową:

```
mosquitto_pub -h localhost -t "test/topic" -m "Wiadomość testowa"
```

Prawidłowe odebranie wiadomości przez subskrybenta potwierdziło, że broker poprawnie zarządza ruchem i jest gotowy do integracji z pozostałymi komponentami systemu, takimi jak Home Assistant czy czujniki IoT.

3.4. Postawienie Home Assistant

3.4.1. Instalacja i konfiguracja platformy Home Assistant

Ostatnim etapem budowy fundamentów infrastruktury serwerowej było wdrożenie platformy Home Assistant. Jest to otwartoźródłowe oprogramowanie służące do centralnego zarządzania automatyką domową, które integruje tysiące urządzeń od różnych producentów w ramach jednego, spójnego interfejsu.

3.4.1.1. Wybór metody instalacji

Zdecydowano się na wersję **Home Assistant Container**. Wybór ten, w przeciwieństwie do wersji **Home Assistant OS** lub **Supervised**, zapewnia pełną kontrolę nad systemem operacyjnym gospodarza oraz pozwala na współdzielenie zasobów z innymi usługami działającymi w kontenerach (takimi jak broker MQTT).

3.4.1.2. Konfiguracja usługi w Docker Compose

Usługa Home Assistant została dodana do infrastruktury kontenerowej. W katalogu głównym projektu utworzono folder `~/homeassistant`, a definicję kontenera umieszczono w pliku `docker-compose.yml`:

```
services:
  homeassistant:
    container_name: homeassistant
    image: "ghcr.io/home-assistant/home-assistant:stable"
    volumes:
      - /home/angelo/homeassistant/config:/config
      - /etc/localtime:/etc/localtime:ro
    restart: always
    privileged: true
    network_mode: host
```

Zastosowanie `network_mode: host` jest kluczowe dla poprawnego działania wielu integracji, szczególnie tych opartych na rozgłaszaniu mDNS (np. wykrywanie urządzeń w sieci lokalnej). Flaga `privileged: true` umożliwia kontenerowi dostęp do portów szeregowych i sprzętu, co może być przydatne przy ewentualnej rozbudowie o adaptory Zigbee lub Z-Wave.

3.4.1.3. Pierwsze uruchomienie i konfiguracja wstępna

Po uruchomieniu kontenera za pomocą `docker-compose up -d`, interfejs webowy stał się dostępny pod adresem IP serwera na porcie 8123. Proces wstępnej konfiguracji (Onboarding) objął:

1. Utworzenie konta administratora.

2. Określenie lokalizacji geograficznej (niezbędne do automatyzacji opartych na wschodzie i zachodzie słońca).
3. Podstawowe skanowanie sieci w celu wykrycia dostępnych urządzeń.

3.4.1.4. Integracja z brokerem MQTT

Kluczowym krokiem z punktu widzenia realizowanego projektu była integracja Home Assistant z wcześniej uruchomionym brokerem Mosquitto. W tym celu:

1. Przejdzono do sekcji **Ustawienia -> Urządzenia oraz usługi**.
2. Dodano nową integrację **MQTT**.
3. Jako adres brokera podano localhost (ze względu na działanie w trybie sieciowym hosta) oraz standardowy port 1883.

Pomyślna konfiguracja integracji pozwoliła na automatyczne wykrywanie urządzeń wspierających standard **MQTT Discovery** oraz umożliwiła ręczne definiowanie encji (czujników i przełączników) w pliku konfiguracyjnym `configuration.yaml`. Stanowi to gotową bazę do podłączenia węzłów opartych na układach ESP32.

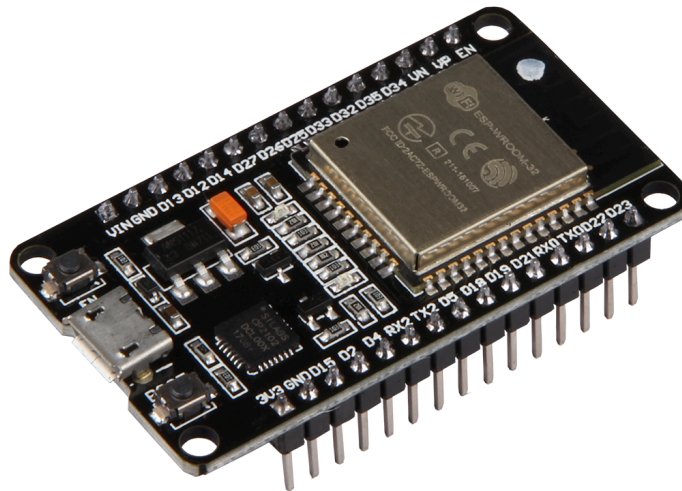
3.5. Implementacja węzłów i analiza ruchu

3.5.1. Implementacja oprogramowania węzła sensorowego (Node-1)

Węzeł nazwany roboczo **Node-1** stanowi kluczowy element wykonawczy warstwy fizycznej systemu. Jest to autonomiczna jednostka, której zadaniem jest generowanie i transmisja danych telemetrycznych do centralnego brokera MQTT. W ramach opisanej fazy projektu, w celu weryfikacji poprawności przepływu danych oraz logiki wizualizacji w systemie Home Assistant, zdecydowano się na zastosowanie programowej symulacji wartości pomiarowych.

3.5.2. Architektura i wybór komponentów

Sercem urządzenia jest układ ESP32 DevKit V1, który został wybrany ze względu na zintegrowany moduł Wi-Fi oraz niskie zużycie energii. Choć docelowo system przewiduje wykorzystanie czujników fizycznych (takich jak BME280), na obecnym etapie zaimplementowano generator sygnałów oparty na funkcjach matematycznych. Takie podejście pozwoliło na:



Rysunek 11: Układ ESP32 DevKit V1

- niezależną weryfikację stabilności połączenia sieciowego i sesji MQTT,
- testowanie reakcji systemu Home Assistant na dynamiczne zmiany wartości w pełnym zakresie,
- eliminację błędów wynikających z potencjalnych problemów sprzętowych na etapie tworzenia szkieletu oprogramowania.

3.5.3. Architektura i logika oprogramowania

Oprogramowanie węzła zostało opracowane w środowisku PlatformIO z wykorzystaniem frameworka Arduino. Główna logika firmware opiera się na maszynie stanów, która zarządza procesami inicjalizacji, nawiązywania połączenia oraz transmisji danych.

Do najważniejszych komponentów programowych należą:

- WiFiManager: Moduł odpowiedzialny za utrzymywanie stabilnego połączenia z lokalną siecią bezprzewodową.
- PubSubClient: Biblioteka obsługująca protokół MQTT w wersji 3.1.1.
- ArduinoJson: Wykorzystywana do serializacji danych pomiarowych do formatu JSON.

3.5.4. Proces przetwarzania i symulacji danych

Algorytm działania węzła w standardowym cyklu pracy obejmuje generowanie wartości przy użyciu funkcji sinus. Pozwala to na uzyskanie płynnych, przewidywalnych zmian parametrów, co jest szczególnie przydatne przy kalibracji wykresów w panelu użytkownika.

Wartość temperatury obliczana jest według wzoru: $T(t) = T_{\text{offset}} + A \cdot \sin(\omega t)$ Gdzie T_{offset} to temperatura bazowa, A to amplituda zmian, a ω określa częstotliwość cyklu.

Algorytm w każdym kroku wykonuje następujące czynności:

1. Łączność: Sprawdzenie stanu połączenia Wi-Fi i brokera MQTT.

2. Generowanie danych: Wyznaczenie kolejnej wartości funkcji sinus dla temperatury oraz analogicznych wartości dla wilgotności i ciśnienia.
3. Publikacja: Dane są pakowane do obiektu JSON i wysyłane na temat: tele/node-1/sensor. Przykład wysyłanej ramki danych:

```
{
  "temperature": 22.5,
  "humidity": 45.2,
  "pressure": 1013.2,
  "uptime": 3600
}
```

4. Oczekiwanie: Węzeł odczekuje zdefiniowany interwał (np. 30 sekund) przed kolejną iteracją.

3.5.5. Integracja z Home Assistant

Pomimo zastosowania danych symulowanych, od strony systemu Home Assistant węzeł jest widziany jako rzeczywiste urządzenie pomiarowe. W pliku `configuration.yaml` zdefiniowano encje typu sensor, które interpretują przychodzące ramki JSON.

```
mqtt:
  sensor:
    - name: "Symulowana Temperatura"
      state_topic: "tele/node-1/sensor"
      unit_of_measurement: "°C"
      value_template: "{{ value_json.temperature }}"
      device_class: temperature

    - name: "Status Grzejnika"
      state_topic: "esp32/actuator/heater"
      payload_on: "ON"
      payload_off: "OFF"
```

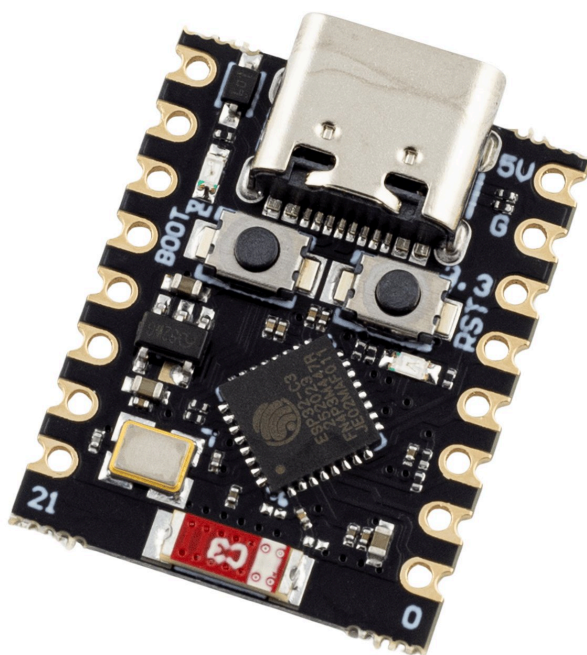
Dzięki takiemu podejściu, dane są natychmiast dostępne dla silnika automatyzacji, umożliwiając np. wizualizację pracy „grzejnika” na tym samym wykresie co temperatura.

3.5.6. Węzeł wykonawczy i silnik autowykrywania (Node-2)

Drugi zaimplementowany węzeł, nazwany **Node-2**, reprezentuje bardziej zaawansowaną klasę urządzeń krawędziowych (**edge nodes**). W przeciwieństwie do Node-1, który skupiał się na teledzieleniu, Node-2 pełni rolę inteligentnego sterownika wykonawczego, integrującego mechanizmy dwukierunkowej komunikacji oraz autonomicznej rejestracji w systemie nadrzędnym.

3.5.7. Architektura sprzętowa

Jako platformę sprzętową dla tego węzła wybrano mikrokontroler ESP32-C3 DevKit, oparty na otwartej architekturze RISC-V. Wybór ten podyktowany był chęcią przetestowania nowoczesnego standardu układów ESP, które oferują zoptymalizowany pobór mocy oraz wsparcie dla najnowszych mechanizmów bezpieczeństwa przy zachowaniu pełnej kompatybilności z ekosystemem Wi-Fi i MQTT.



Rysunek 12: Mikrokontroler ESP32-C3 wykorzystany w module Node-2

3.5.8. Separacja tematów: Polecenia a stan (Command vs State)

Kluczowym elementem logiki komunikacyjnej Node-2 jest pełna separacja tematów sterujących od tematów raportujących stan. Zastosowano architekturę dwukierunkową, aby zapobiec pętlom logicznym oraz zagwarantować bezwzględną synchronizację między stanem faktycznym sprzętu a interfejsem użytkownika.

W systemie zdefiniowano dwa główne kanały dla każdej encji wykonawczej (np. przekaźnika):

- Temat poleceń (`esp32/actuator/set`): Służy wyłącznie do przesyłania żądań zmiany stanu z Home Assistant do węzła.
- Temat stanu (`esp32/actuator/state`): Służy do raportowania przez węzeł aktualnego potwierdzonego stanu urządzenia.

Dzięki takiemu podejściu, interfejs użytkownika aktualizuje status urządzenia dopiero po otrzymaniu potwierdzenia z mikrokontrolera. Eliminuje to błędy typu „ghost switching”, gdzie ikona w aplikacji zmienia stan mimo braku fizycznej reakcji urządzenia (np. z powodu chwilowej utraty zasięgu Wi-Fi).

3.5.9. Mechanizm MQTT Self-Discovery

Węzeł Node-2 implementuje standard Home Assistant MQTT Discovery, co pozwala na całkowitą eliminację konieczności ręcznej edycji plików konfiguracyjnych w systemie nadrzędnym. Natychmiast po uruchomieniu i nawiązaniu połączenia z siecią, mikrokontroler wysyła ramkę konfiguracyjną w formacie JSON na specjalny temat odkrywania (`homeassistant/switch/esp32_heater_actuator/config`).

Poniżej przedstawiono przykładowy ładunek danych (payload) generowany przez węzeł podczas procesu autowykrywania:

```

{
  "name": "Heater Actuator",
  "unique_id": "esp32_heater_actuator",
  "command_topic": "esp32/actuator/set",
  "state_topic": "esp32/actuator/state",
  "payload_on": "ON",
  "payload_off": "OFF",
  "state_on": "ON",
  "state_off": "OFF",
  "icon": "mdi:radiator",
  "device": {
    "identifiers": ["esp32c3_8291AB"],
    "name": "Node-2 Actuator Engine",
    "model": "ESP32-C3 DevKit",
    "manufacturer": "Espressif"
  }
}

```

Dzięki przesłaniu metadanych takich jak model urządzenia czy producent, Home Assistant automatycznie tworzy nie tylko samą encję sterującą, ale również grupuje ją w ramach logicznego urządzenia. Ułatwia to zarządzanie systemem w miarę dodawania kolejnych modułów wykonawczych.

3.5.10. Logika operacyjna i synchronizacja

Proces obsługi polecenia przez Node-2 przebiega w następujących krokach:

1. Odbiór: Węzeł subskrybuje temat `esp32/actuator/#`. Po otrzymaniu wiadomości "ON", następuje wysterowanie odpowiedniego pinu GPIO.
2. Weryfikacja: System sprawdza stan logiczny wyjścia.
3. Potwierdzenie: Węzeł publikuje aktualny stan na temat `esp32/actuator/state`.

Zastosowanie flagi `Retain` dla komunikatów stanu pozwala na natychmiastowe odzyskanie poprawnego statusu urządzenia przez Home Assistant nawet po restarcie serwera, co zwiększa ogólną niezawodność całego ekosystemu IoT.

3.5.11. Automatyzacja i zcentralizowane zarządzanie stanem

W zaimplementowanej architekturze modularnej kluczowym aspektem jest zcentralizowane sterowanie logiką systemu. Zamiast bezpośredniej komunikacji między węzłem czujnika a węzłem wykonawczym (typ **peer-to-peer**), oba moduły komunikują się wyłącznie za pośrednictwem systemu Home Assistant poprzez protokół MQTT. Home Assistant pełni tutaj rolę jedyne go źródła prawdy (**Single Source of Truth**), co pozwala na zachowanie spójności stanu całego ekosystemu.

Przepływ danych i logiki sterowania odbywa się w następujący sposób:

1. Publikacja danych: Węzeł czujnika (**Sensor Node**) publikuje odczyty temperatury na temat MQTT `esp32/sensor/temperature`.
2. Przetwarzanie logiki: Home Assistant subskrybuje ten temat, analizuje otrzymaną wartość i porównuje ją ze zdefiniowanym progiem. Na tej podstawie system decyduje o zmianie stanu encji `switch.heater_actuator`.

3. Sterowanie urządzeniem: Zmiana stanu encji w Home Assistant powoduje wysłanie odpowiedniego polecenia do węzła wykonawczego (**Node-2**). Węzeł ten nasłuchuje komunikatów sterujących i reaguje wyłącznie na instrukcje pochodzące z systemu nadrzędnego.
4. Interakcja użytkownika: Dzięki centralizacji zmiana stanu urządzenia może nastąpić również ręcznie poprzez przełącznik w interfejsie graficznym (**Dashboard Switch**). W takim przypadku Home Assistant aktualizuje stan encji `switch.heater_actuator`, co automatycznie propaguje się do fizycznego urządzenia.

Takie podejście gwarantuje, że logika sterowania jest oddzielona od warstwy sprzętowej, co ułatwia modyfikację algorytmów automatyzacji bez konieczności przeprogramowywania poszczególnych mikrokontrolerów.

4. Podsumowanie

Bibliografia